

From Stigmergic Module Routing to a Continual World Model: Seventeen Preregistered Experiments, Their Negative Results, and a Planning Gap That Latent Error Does Not Explain

Hassan Akaou

Independent researcher, France

<https://github.com/sahnas/stigmergic-module-routing>

October 2026

Abstract

We report a preregistered study that follows one idea from a synthetic compositional world to a released world model, and keeps its negative results. The idea: small learning modules coordinated by stigmergic traces (reinforced by use, evaporated when the task symbol is solicited), with the least committed module recruited when one fails. In eleven toy experiments, with the mechanism never tuned and opponents tuned on separate seeds, the mechanism gives resilience to module loss without a routing gradient, on par with sliding-window and discounted UCB; it does not beat routers trained by gradient, dense or sparse; per-module alarm thresholds bring nothing; and it discovers no compositional structure unless each module receives its own local error, in which case trace routing finds the shared structure with a cleaner decomposition than dense or sparse gradient routers. Five further experiments show that the mechanism protects no function by itself: once no spare capacity is left, a specialist is consumed under every recruitment rule, and only reserves held back until an event preserve it. On the frozen representation of the released LeWorldModel for Two-rooms, with a scenario of changing dynamics, banks of specialists with freezing and held-back reserves are not better than a reservoir replay buffer of 20,000 windows on any retention measure, which stops the line by the preregistered rule. On the way we found that LeWorldModel’s own predictor, retrained from scratch on the frozen released encoder, reaches ten times lower latent prediction error on held-out expert trajectories and plans far worse (36% vs 85% success on Two-rooms, 100 episodes each); five diagnostics exclude action normalisation, one-step error on random actions, open-loop compounding, candidate ranking and wall crossing, and leave one measured anomaly, a 27-fold optimism of the plan chosen by the cross-entropy solver. In-distribution latent error, with every offline refinement we tried, does not predict closed-loop planning. Code, data, preregistrations, code hashes, seed ledgers and a provenance report are public.

1 Introduction

Modular neural systems need a way to decide which module handles which part of a task. The dominant answer is a router trained jointly with the modules by gradient descent, as in mixtures of experts (Jacobs et al., 1991; Shazeer et al., 2017). Another family routes with reinforcement learning (Rosenbaum et al., 2018), and a third coordinates modules through a shared workspace (Goyal et al., 2022); see Pfeiffer et al. (2023) for a survey. In some settings, however, the router cannot receive gradients: modules may be physically separated, heterogeneous, added or removed during operation, or simply not differentiable end to end. Such settings also raise a robustness question: what happens when a module fails?

Social insects solve a related problem without a central controller. Ants coordinate through traces deposited in the environment, a mechanism called stigmergy (Grassé, 1959), which inspired

ant colony optimisation (Dorigo et al., 1996) and ant-based network routing (Di Caro and Dorigo, 1998). Division of labour emerges from individual response thresholds (Bonabeau et al., 1996; Theraulaz et al., 1998), and recruitment of idle workers has been transferred to cooperative robots (Krieger et al., 2000).

We ask whether these ingredients, transposed to routing between modules that learn during operation, produce a useful coordination mechanism, and where it breaks. The mechanism combines evaporating traces on symbol-to-module edges, an alarm on abnormal quality drops, and recruitment of the least committed module. We test it in a deliberately small synthetic world, using preregistered hypotheses (Nosek et al., 2018), separate development and test seeds, tuned opponents and an untuned mechanism.

Our contributions are empirical and include negative results, reported at the same rank as the positive ones. (i) In its most defensible form, the mechanism is a distributed backup-capacity heuristic based on the historical commitment of modules: resilience comparable to good non-stationary bandits without a routing gradient, no advantage over routers trained by gradient, no measurable role for per-module thresholds. (ii) It discovers no compositional structure unless each module has its own local error; with that error, hard assignment by traces yields a cleaner decomposition than dense or sparse gradient routers. (iii) It protects no function by itself: once spare capacity is gone, specialists are consumed by exploration and recruitment alike, and only held-back reserves preserve them. (iv) On the frozen representation of a released world model, under changing dynamics, banks of specialists are not worth more than reservoir replay, and the line stops. (v) A predictor retrained on that frozen encoder has ten times lower latent error and plans far worse; five diagnostics narrow the cause without finding it, which we report as an open question to the model’s authors. The study also documents, as part of its method, the provenance problems of AI-assisted experimentation (Section 9).

2 Related work

Stigmergy and division of labour. Ant colony optimisation uses pheromone deposit and evaporation to bias the construction of solutions (Dorigo et al., 1996); AntNet applies it to packet routing in communication networks (Di Caro and Dorigo, 1998). Response-threshold models explain how heterogeneous thresholds yield division of labour and reallocation after worker loss (Bonabeau et al., 1996; Theraulaz et al., 1998). Krieger et al. (2000) showed that ant-like task allocation and recruitment improve the foraging efficiency of groups of robots. In these works, the agents do not learn new functions during operation.

Modular networks and routing. Mixtures of experts route inputs with a gating network trained by gradient (Jacobs et al., 1991; Shazeer et al., 2017). Routing networks select function blocks with a router trained by reinforcement learning (Rosenbaum et al., 2018), which is the closest structural precedent to our setting. Shared global workspaces coordinate specialised modules through a limited-capacity bottleneck (Goyal et al., 2022). The Artificial Neural Tissue of Thangavelautham and D’Eleuterio (2012) regulates modules with diffusing chemical-like signals and obtains self-organised task decomposition, but learns through evolution over many generations rather than during operation. Bahdanau et al. (2019) showed that modular networks generalise systematically when their layout is right, but that learning the layout is hard, a finding that our last experiment echoes.

Non-stationary bandits. Routing without gradient can be cast as a bandit problem per task symbol, non-stationary because modules learn and fail. Sliding-window and discounted UCB (Garivier and Moulines, 2011) and change-detection approaches such as M-UCB (Cao et al., 2019) are natural opponents and are used as such here.

Pheromone routing between AI agents. Recent work routes queries between LLM-based agents with pheromone memories, including fault-aware adaptation from execution feedback (Du et al., 2026) and task-specific pheromone specialists (Wang et al., 2026). There, agents are fixed specialists: failed agents are avoided, not replaced by an agent that relearns their function. The specific combination studied here (modules that learn together with the routing, solicitation-triggered forgetting, and load-aware recruitment of a module that relearns a lost function) was not found in these works. This is based on targeted searches, not an exhaustive review.

3 Mechanism

A pool of modules $m \in \{1, \dots, M\}$ shares a set of task symbols s . A task is a path of one or two symbols; each symbol is served by one module, and the modules are applied in sequence. Modules are small regressors $f_m(x) = W_mx + b_m + g_m(x)$, where g_m is a one-hidden-layer ReLU network with 32 units. They communicate through the observation space, a fixed interface, and learn by gradient descent (Adam, learning rate $3 \cdot 10^{-3}$, batches of 32) along the path actually used.

Traces and selection. Each edge (s, m) carries a trace $\tau_{s,m}$, initialised at 0. The module serving symbol s is sampled with probability

$$P(m \mid s) = \frac{\exp(\tau_{s,m}/T)}{\sum_{m'} \exp(\tau_{s,m'}/T)}, \quad T = 0.05, \quad (1)$$

over available modules, excluding the module already used in the first slot of the path. After each batch, the path receives the quality $q = \text{clip}(1 - \text{MSE}/\text{Var}(y), 0, 1)$, a clipped coefficient of determination.

Solicitation-triggered evaporation. For every symbol s solicited by the batch, all its traces evaporate, then the edge used receives a deposit:

$$\tau_{s,\cdot} \leftarrow (1 - \rho) \tau_{s,\cdot}, \quad \tau_{s,m} \leftarrow \tau_{s,m} + \rho q, \quad \rho = 0.02. \quad (2)$$

The trace of a repeatedly used edge thus tracks a moving average of its quality, while the other edges of a solicited symbol fade. Edges of symbols that are not solicited are left untouched. We compare this rule (*aco*) with three alternatives: a cumulative mean of q on the edge used, with no forgetting (*cumulative*); a moving average on the edge used only (*ema*, forgetting by recency); and evaporation of all traces of all symbols at every step with the same rate ρ (*global*, literal time-based evaporation).

Alarm and recruitment. Each module keeps exponential moving averages (rate 0.05) of its quality $\bar{q}_m$ and of its absolute deviation d_m . After at least 50 uses, if a batch gives $q < \bar{q}_m - \kappa \max(d_m, 0.02)$ with $\kappa = 3$, and symbol s is not in a 100-step cooldown, an alarm fires. The module r with the lowest total trace $\sum_{s'} \tau_{s',r}$ among available modules off the current path is recruited. We call this total trace the module’s load: it measures how much the routing already relies on the module, not its computational load, and the recruit is therefore the least committed module: its trace for s is raised to the incumbent’s level, $\tau_{s,r} \leftarrow \max(\tau_{s,r}, \tau_{s,m})$, and its statistics are reset. The recruit then competes with the incumbent through ordinary selection and learning. The full mechanism, evaporation plus alarm plus recruitment, is denoted *aco_thresholds*.

What the traces add, and what "stigmergy" does not mean here. Without the stigmergic reading, the mechanism is a table of affinities per (symbol, module), a decay applied when a symbol is used, an anomaly detector and a failover policy. Two of its parts are nevertheless specific. First, the decay also hits the unused edges of a solicited symbol, which a plain moving average of the quality of the edge used (*ema*) does not do; this difference is measurable (Experiment 3). Second, the load used for recruitment is read from the same traces, so the failover policy consumes the routing memory rather than a separate signal. The word stigmergy should not be over-read: there is no environment in which modules act, only a shared table. Its hyperparameters were chosen a priori and never tuned.

4 Experimental setup

World. Five fixed primitives act on $\mathbb{R}^8$: $P_k(x) = Q_k x + 0.5 \tanh(B_k x)$, with Q_k a random orthogonal matrix and B_k a random Gaussian matrix with entry variance $1.5^2/8$. A task is a primitive or an ordered pair (i, j) , $y = P_j(P_i(x))$, with $x \sim \mathcal{N}(0, I)$. Of the 20 ordered pairs, 12 are used for training (each primitive appearing in both positions) and 8 are held out for zero-shot evaluation. The world is regenerated for each seed.

Opponents. Depending on the experiment: a monolithic MLP (two hidden layers of 128 units) receiving one-hot task symbols; fixed orchestration (symbol k to module k); a router trained by gradient (softmax mixture of all modules per symbol, mixture-of-experts style); a REINFORCE router (Williams, 1992) in the spirit of routing networks; SW-UCB, D-UCB and M-UCB; an independent change-point UCB with per-arm reset; and sparse top- k mixtures of experts with noisy gating (Shazeer et al., 2017). Opponents were tuned on development seeds; the tested mechanism was not.

Protocol and statistics. Each test was preregistered before running its test seeds, with hypotheses, thresholds and seeds written in advance (see Section 9 for exceptions). Comparisons are paired across seeds with two-sided Wilcoxon signed-rank tests (Wilcoxon, 1945), Bonferroni-corrected within each family of hypotheses; we report mean differences with 95% bootstrap confidence intervals (10,000 resamples). Everything runs on a single CPU core.

5 Results

5.1 Experiment 1: compensation with given symbols (seeds 0–9)

Eight modules learn 17 tasks presented in successive blocks (two passes, 150 steps per block). The module carrying one primitive k^* then dies permanently (zero output, no relearning), followed by a short recovery phase (two passes, 60 steps per block). Table 1 and Figure 1 summarise the results.

Forgetting enables compensation: evaporation recovers while traces without forgetting and fixed orchestration never do (difference $+0.92$, 10/10 seeds, $p = 0.002$). Alarm and recruitment reduce rerouting latency from about 555 to 4.5 steps and improve recovery (10/10 seeds, $p = 0.002$). At the same rate, time-based evaporation destroys retention (-0.76 , 10/10, $p = 0.002$). Evaporation beats the monolithic MLP in zero-shot generalisation but not the gradient-trained router ($p = 0.43$), and does not differ from forgetting by recency ($p = 0.56$ and $p = 1.0$). An apparent advantage of the full mechanism over the gradient-trained router (zero-shot 0.88 vs 0.73, 9/10 seeds) was exploratory.

Variant	Retention	Zero-shot	k^* tasks, recovered	Rerouting
Monolithic MLP	-0.47	-0.50	-0.05	n/a
Fixed orchestration	0.94	0.91	0.00	never
Router trained by gradient	0.81	0.73	0.98	324
No forgetting (cumulative)	0.75	0.70	0.00	never
Forgetting by recency (ema)	0.76	0.68	0.75	524
Evaporation (aco)	0.83	0.72	0.92	555
Time-based evaporation	0.07	-0.09	0.13	37
Evaporation + alarm + recruitment	0.91	0.88	0.97	4.5

Table 1: Experiment 1, means over 10 seeds (rerouting latency: median). Values are R^2 .

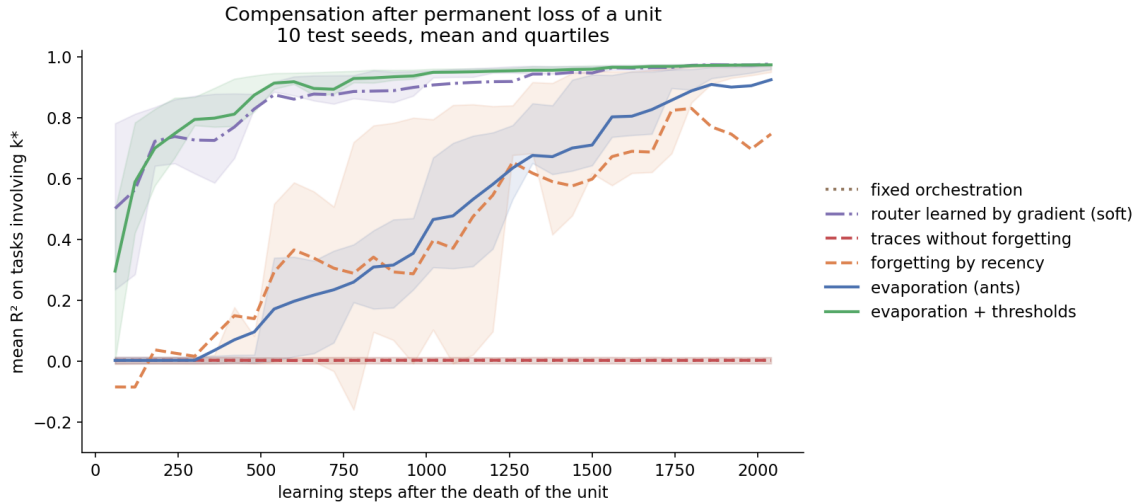


Figure 1: Recovery after the permanent death of the module carrying k^* (Experiment 1, mean and interquartile range over 10 seeds).

5.2 Experiment 2: confirmation (seeds 20–29)

That exploratory advantage was not confirmed on fresh seeds: zero-shot $+0.008$ (95% CI $[-0.09, +0.09]$, $p = 0.63$) and retention $+0.014$ ($[-0.04, +0.07]$, $p = 0.56$). The first sample was unfavourable to the gradient-trained router (mean retention 0.81 versus 0.88 on the second). With gradients available, the mechanism ties with a gradient-trained router.

5.3 Experiment 3: no routing gradient (seeds 40–59)

The routing now receives no gradient. After initial learning, a production phase applies four events: the death of a module; the arrival of two new modules together with another death; a change in the definition of one primitive; and a simultaneous double death. The pool has ten slots, seven occupied at the start, and the system is never told which modules are dead. The main measure, availability, is the mean R^2 over the 17 tasks evaluated after every 60-step block of the production phase. A 2×2 design crosses the forgetting rule (ema, aco) with the alarm and recruitment (absent, present).

The full mechanism beats recency by $+0.35$ ($[+0.29, +0.40]$, 20/20 seeds); the alarm and recruitment add $+0.14$ (18/20, $p = 0.0001$); evaporation beats recency by $+0.21$ (20/20; no sign was predicted). The advantage is not only an initial-learning effect: on the seven seeds where both systems start healthy, the mechanism loses 0.30 of R^2 across the events, recency 0.77 (Figure 2).

Variant	Availability	Collapses	Final R^2	Final zero-shot
Recency (ema)	0.41	10/20	0.22	0.07
Recency + alarm + recruitment	0.51	7/20	0.61	0.45
Evaporation (aco)	0.57	4/20	0.37	0.20
Evaporation + alarm + recruitment	0.76	0/20	0.60	0.39

Table 2: Experiment 3, means over 20 seeds. Collapses: seeds with initial retention below 0.5.

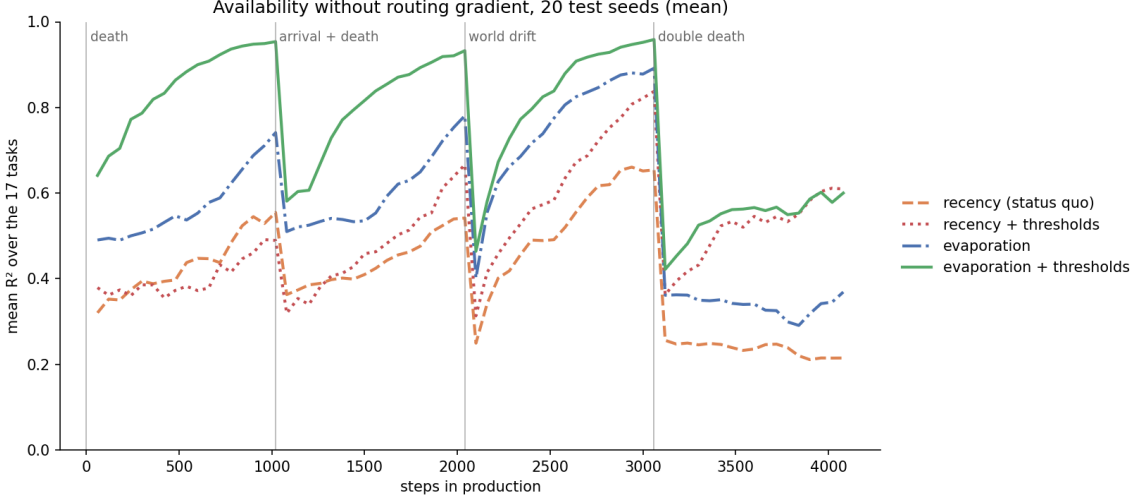


Figure 2: Availability during the production phase without routing gradient (Experiment 3, mean over 20 seeds). Vertical lines mark the four events.

5.4 Experiment 4: standard non-stationary bandits (seeds 80–99; replication 60–74)

The same scenario is run against SW-UCB, D-UCB and M-UCB, each tuned over its own grid on development seeds. The preregistered thesis that the mechanism beats non-stationary bandits required all three comparisons to succeed.

Variant	Availability	Initial R^2	Final R^2
Evaporation + alarm + recruitment	0.74	0.94	0.65
SW-UCB (window 50)	0.72	0.69	0.86
D-UCB ($\gamma = 0.98$)	0.72	0.66	0.88
M-UCB (change detection)	0.59	0.90	0.72

Table 3: Experiment 4, main test, means over 20 seeds.

Against SW-UCB the difference is +0.02 ($[-0.02, +0.06]$, $p = 0.39$) and against D-UCB +0.03 ($[-0.01, +0.06]$, $p = 0.20$): neither is confirmed. Against M-UCB it is +0.15 (20/20 seeds). The replication on 15 other seeds shows the same pattern, with the first two differences positive but above the preregistered threshold ($p = 0.03$). The thesis is therefore not retained. The profiles differ: the mechanism learns better initially and absorbs the first events better, whereas SW-UCB and D-UCB recover better late in the run, notably after the double death.

5.5 Experiment 5: a tuned change-point bandit (seeds 60–79)

An independent change-point UCB (sliding window, per-arm reset on a quality drop), tuned over 16 configurations, is clearly stronger than recency (+0.16, 18/20) but weaker than the mechanism

(0.77 vs 0.61, difference +0.16, $[+0.11, +0.22]$, 18/20, $p < 10^{-4}$). It learns better initially (0.95 vs 0.91) but loses more across events (0.37 vs 0.21). This is consistent with the result against M-UCB.

5.6 Experiment 6: ablation (seeds 80–99)

Recruiting a random available module instead of the least committed one lowers availability from 0.74 to 0.52 (+0.22 for least-committed recruitment, $[+0.19, +0.27]$, 20/20). Replacing per-module alarm thresholds with a single collective threshold has no detectable effect (+0.002, $[-0.03, +0.03]$, $p = 0.78$). The active ingredient is therefore a piece of collective information, the load of each module in the sense of its total trace, read at the moment of failure. In this scenario all modules are identical, which may explain why individual thresholds bring nothing.

5.7 Experiment 7: opaque task identifiers (seeds 600–609)

Each composition now receives only an opaque identifier. The system knows that a path has two slots but not which primitives a task combines; with 8 modules and 12 tasks, success requires discovering that tasks share primitives. After interleaved learning of the 12 known compositions, each of the 8 new compositions is learned for 200 steps from the same state.

Variant	Known	Speed	Final new	Interf.	Align.
Monolithic MLP	0.95	0.96	0.98	0.60	n/a
Router trained by gradient	0.91	0.89	0.97	2.23	−8.8
REINFORCE router	0.13	0.29	0.50	0.21	−0.24
Evaporation + alarm + recruitment	0.21	0.29	0.55	0.21	−0.20

Table 4: Experiment 7, means over 10 seeds. Known: R^2 on the 12 known tasks. Speed: mean R^2 on a new task over 10 evaluations during 200 steps. Final new: R^2 on the new task at the end. Interf.: drop of mean R^2 on the known tasks. Align.: best R^2 of any module approximating each primitive, averaged over primitives.

The mechanism learns new compositions far more slowly than the monolithic MLP and the gradient-trained router (0/10 seeds, $p = 0.002$) and ties with the REINFORCE router. Its lower interference is not a virtue, since little was learned in the first place. No variant aligns its modules with the primitives: none discovers the decomposition. The MLP and the gradient-trained router learn each task as its own function and overwrite previous ones, an instance of catastrophic interference (McCloskey and Cohen, 1989). These failures were visible on the development seeds and were declared in the preregistration before the test.

5.8 Experiment 8: sparse top- k mixtures of experts (seeds 700–719)

This experiment and the next were run after a first version of this manuscript. In the protocol of Experiment 1, the mechanism faces routers trained by gradient with noisy top- k gating, per task symbol. The noise scale and an optional balancing loss were tuned on development seeds. As declared in the preregistration, two opponents were kept after seeing the tuning results, because the best top-1 configuration never recovers from the loss of a module, which would have made the compensation comparison trivial.

Against top-2, the mechanism is behind in zero-shot generalisation (−0.03) and retention (−0.04), not significantly at the preregistered threshold ($p = 0.009$ and 0.012 against 0.0083), and ahead in recovery by a negligible +0.011 (18/20, significant). Against top-1, it is clearly behind in zero-shot generalisation (−0.08) and retention (−0.07), and far ahead in recovery (+0.93, 20/20): a top-1 router cannot reroute once its selected module is dead. With gradients available, the mechanism brings no learning advantage.

Variant	Retention	Zero-shot	k^* tasks, recovered
Evaporation + alarm + recruitment	0.89	0.87	0.97
Top-1 (noise 0.3, balancing 0.01)	0.96	0.95	0.05
Top-2 (noise 0.3)	0.93	0.89	0.96

Table 5: Experiment 8, means over 20 seeds.

5.9 Experiment 9: heterogeneous modules (seeds 720–739)

The production scenario of Experiment 3 is rerun with heterogeneous modules: their hidden sizes are 2, 4, 8, 16, 32, 64, 2, 4, 8 and 16 units, in a seed-dependent order, so that a little-used module may be little used because it is weak. The bandits keep the configurations of Experiment 4, as preregistered; nothing is tuned.

Variant	Availability	Initial R^2	Collapses	Final R^2
Evaporation + individual thresholds	0.72	0.89	0/20	0.55
Evaporation + collective threshold	0.72	0.89	1/20	0.53
SW-UCB (window 50)	0.71	0.75	2/20	0.76
D-UCB ($\gamma = 0.98$)	0.68	0.63	7/20	0.74

Table 6: Experiment 9, means over 20 seeds.

Individual thresholds still bring nothing (+0.002, $[-0.02, +0.03]$, $p = 0.93$), contrary to the hypothesis that heterogeneity would make them useful. The mechanism ties with SW-UCB (+0.01, $p = 0.90$) and is ahead of D-UCB (+0.04, $[+0.01, +0.07]$, 16/20, $p = 0.014$). That last edge should not be read as structural: the bandits were tuned for homogeneous modules, and D-UCB collapses at initial learning on 7 of 20 seeds. The profiles of Experiment 4 persist, with better initial learning for the mechanism and better final recovery for the bandits.

5.10 Experiments 10 and 10b: oracle local credit (seeds 800–809 and 810–819)

The diagnosis of Experiment 7 is put to the test in the same opaque-identifier world. An oracle gives each slot of the path its own local quality: slot 1 is compared with the true intermediate $P_i(x)$, slot 2 with y . Level 1 uses the local quality for traces and alarms only, modules still learning on the global loss; level 2 also gives each module its own learning target. Experiments 10 and 10b are two independent implementations of this design, written without knowledge of each other (Section 9) and differing slightly in the oracle for slot 2; on three shared seeds the reference values are identical and the local variants agree within 0.04, except one known-task value that differs by 0.10 (seed 802). Experiment 10b adds a dense gradient router given the same local losses.

Exp.	Variant	Known	Speed	Interf.	Align.
10	Reference (global credit)	0.18	0.31	0.21	−0.24
10	Level 1: local credit, routing only	0.09	0.22	0.19	−0.27
10	Level 2: local credit, routing + content	0.97	0.94	0.20	0.99
10b	Reference	0.20	0.30	0.22	−0.28
10b	Level 2	0.97	0.95	0.16	0.99
10b	Dense gradient router, local losses	0.99	0.95	0.66	−1.65

Table 7: Experiments 10 and 10b, means over 10 seeds each. Columns as in Table 4.

Local credit for routing alone makes things worse (known tasks −0.09 and speed −0.09, 0/10 seeds each): the traces favour modules that resemble primitives while the global loss trains them

towards something else. Level 2 learns the new compositions far faster than the reference (+0.63, 10/10 in Experiment 10; +0.65, 10/10 in 10b), with modules aligned with the primitives. Given the same local losses, the dense gradient router ties on speed (-0.008 , $p = 0.85$) but smears the modules (alignment -1.65 vs 0.99 , 10/10) and interferes with the known tasks (0.66 vs 0.16 , 10/10). The lock of Experiment 7 was therefore the learning signal of the modules, not an inability of the traces to find a path once modules are correct.

5.11 Experiment 11: sparse routers with local losses (seeds 820–829)

Is the clean decomposition of 10b a property of traces or of hard assignment in general? Top-1 and top-2 routers, with the configurations of Experiment 8 and the same local losses, are compared with trace routing. The prior stated before the experiment gave a 60% chance that a sparse router would align as well; after the development seed it was revised to 35%, as recorded in the preregistration.

Variant	Known tasks	Speed on new	Interference	Alignment
Trace routing, local losses	0.97	0.95	0.19	0.99
Top-1 router, local losses	0.97	0.66	0.28	0.46
Top-2 router, local losses	0.99	0.87	1.21	0.94

Table 8: Experiment 11, means over 10 seeds.

Neither sparse router matches the traces. Top-1 gating learns new compositions slowly (-0.29 vs traces, 10/10) and aligns modules only partially (-0.53 , 10/10), with no significant difference in interference. Top-2 gating aligns almost as well (-0.05 , 10/10) but interferes far more (1.21 vs 0.19 , 10/10) and learns somewhat slower (-0.08 , 10/10). The combination of fast learning, clean alignment and low interference is specific to the trace rule among the routers tested.

6 Beyond the toy world: preservation, modular JEPA, and a continual world model

The experiments of this section were run after the ones above, each preregistered before its test seeds; the preregistrations, with the development runs they declare, are in the repository.

6.1 Experiments 14 to 16b: does the mechanism preserve a function?

A reviewer asked whether the traces select important skills or merely frequently solicited ones. We made one primitive rare (its tasks sampled with relative weight 0.02) in the production world of Experiment 3, killed the module carrying a frequent primitive, and measured the rare skill before and after. Experiment 14 had three protocol flaws found by an external review (reserve modules active from the start, recruitment rules acting during initial learning, recruitments counted from the start); it is kept with its flaws recorded and was redone as Experiment 14b on a common-state harness: one model learned up to the event and deep-copied before every condition, production data drawn once and shared, recruitments timestamped.

With no spare capacity the recruitment rule changes nothing (-0.01 on availability, $p = 0.77$): the rare skill collapses with and without recruitment, consumed by exploration itself, and the first post-event recruit is the rare specialist in 3/10 seeds (random: 2/10). Held-back reserves preserve it (0.89 to 0.92) and recover the killed function; spares active from the start preserve what they had but had learned the rare skill less well (0.59), because they dilute exploration during learning. Experiment 15b (identical relearning streams) found that a consumed specialist relearns its function no faster than a fresh module ($+0.06$ on the area under the learning curve, $p = 0.32$), and that a module repurposed to another function relearns slower than a fresh one

Capacity	Rule	Avail.	Rare before	Rare after	Killed recov.
none (5 active)	least comm.	0.46	0.89	−0.54	0.59
none	random	0.41	0.89	−0.33	0.54
none	none	0.47	0.89	−0.54	0.58
spare (7 active)	least comm.	0.76	0.59	0.61	0.92
dormant (5+2 held)	least comm.	0.87	0.89	0.92	0.96

Table 9: Experiment 14b, seeds 1400–1409, means over 10 seeds.

(−0.20, 0/10, $p = 0.002$). Experiment 16b found no gain from resetting a recruited module’s optimizer state (−0.01) or its weights (−0.06, 1/10). A perturbed replay of all 110 runs of 14b/16b (initial weights perturbed by 10^{-6}) left the six preregistered decisions unchanged while individual runs moved, which is why we read these results at the level of the aggregate.

6.2 Experiments 12 and 13: competing JEPA predictors

In a sequential world with five unnamed primitives that persist for a few steps and eight noise channels per observation, a tiny JEPA (shared encoder, EMA target) hosts competing predictors assigned by hindsight error, as in COMET (Lei et al., 2024) and MOSAIC (Haruno et al., 2001), against an attention-based assignment in the style of Recurrent Independent Mechanisms (Goyal et al., 2021). Competition reaches a module purity of 0.84 against 0.36 for attention-based assignment (5/5 seeds); the prior over the winner needs memory, not the current state (after a switch to a held-out pair, a persistence prior recovers to 0.05 where a state classifier stays at 1.52); traces tie with plain persistence (+0.002). When a sixth primitive appears, surprise-triggered recruitment of a dormant reserve instantiates a new predictor without degrading the old ones (error on the new primitive 0.49 vs 0.68 for fixed competition, 5/5), but no better than competition merely over-provisioned with three extra modules (differences of 0.002 and 0.001).

6.3 Experiment 17: continual dynamics on LeWorldModel’s frozen representation

LeWorldModel (Maes et al., 2026) is a JEPA world model trained end to end from pixels. We kept the released encoder and projector for Two-rooms frozen, cached the 192-dimensional embeddings of all 920,809 frames of its dataset (parity with the model’s own encoding exact in fp32), and trained predictors with LeWorldModel’s own architecture on windows aligned with its configuration (history 3, frameskip 5, action blocks of 5). Regime A is the original dynamics; regime B uses the same images with the action vector rotated by 90° , so that a regime can be inferred only from prediction errors, never from appearance. The stream has three phases of 1500, 1500 and 500 steps of four episodes: A, then B with A at 2%, then A again. Measures are relative five-step latent errors on held-out episodes; access error uses the module chosen before observing the outcome. Three development runs on one seed, declared in the preregistration, set the banks’ thresholds (freezing at 0.1, surprise at four times the recent error after a warm-up, a 100-step cooldown); the replay buffer was not tuned.

System	A, P1	A during B	B, P2	A, P3	B, P3
single predictor	0.070	1.60	0.063	0.058	1.68
single + replay (20k windows)	0.064	0.131	0.080	0.072	0.107
bank, persistence prior	0.108	0.132	0.116	0.111	0.138
bank, traces + recruitment	0.108	0.132	0.116	0.111	0.138

Table 10: Experiment 17, five seeds; standard deviations between 0.002 and 0.047. The banks use twice the forward passes.

The banks behaved as designed on every seed (the A specialist froze before B, one surprise, one activation, no recruitment) and are identical to the fourth decimal, the prior having nothing to decide with one module per regime. Against replay they are not better on any retention measure (A during B: +0.000, 2/5; B at the end: +0.031, 0/5; A at the end: +0.039, 0/5), and they cap A at the level where it froze. By the preregistered stop rule, the trace and recruitment line stops for continual world models.

7 A planning gap that latent error does not explain

The retrained predictor stack (action encoder, predictor, projection head, instantiated from the released configuration at random init and trained 8 epochs on the frozen embeddings) reaches a relative five-step latent error of 0.006 on held-out episodes, against 0.067 for the released predictor; the released one fine-tuned two epochs reaches 0.007. With the authors’ evaluation script unchanged, on 100 episodes each, the released model succeeds in 85 (exact 95% interval 77 to 91) and the retrained one in 36 (27 to 46). Five diagnostics, each a small kernel on a free GPU, failed to explain it:

1. action normalisation (training-frame or dataset statistics): no effect, for either model;
2. one-step error on 8,500 windows of random actions played in the environment: both predictors degrade to the same level (released 0.60, retrained 0.55);
3. open-loop rollouts over ten horizons of five frames, feeding predictions back: the retrained predictor stays better at every horizon, on expert and on random trajectories;
4. ranking of 19 random candidate plans from 40 cloned starts: Spearman correlation between predicted and true cost 0.98 for the retrained, 0.90 for the released;
5. the 100 evaluation episodes split by whether the goal lies across the wall: the gap is as large within a room (40% vs 90%) as across it (19% vs 67%).

One measurement differs: with the authors’ cross-entropy solver reproduced (300 samples, 30 iterations, 30 elites) from 30 cloned starts, the plan it selects is predicted by the retrained model to cost 0.68 and costs 37.0 in truth, a 27-fold optimism, against 12.4 predicted and 43.8 true for the released model; the two executed plans are equally good ($p = 0.75$). The optimizer finds plans the retrained model believes near-perfect. How this optimism turns into closed-loop failure within the authors’ receding-horizon loop is the open question we put to them. What the diagnostics establish is narrower and firm: in-distribution latent error, and every offline refinement of it we tried, does not predict closed-loop planning success.

8 Discussion

What the mechanism is. Across experiments, the mechanism behaves as a coordination and resilience device. It needs forgetting, and at the rate tested, forgetting tied to solicitation: time-based evaporation at the same rate erases routes for symbols that are simply not being used, while a much slower time-based rate was not tested. Its resilience comes from recruiting the least committed module, which avoids disturbing specialists; this echoes the reallocation of idle workers in insect societies (Bonabeau et al., 1996; Krieger et al., 2000). The individuality of alarm thresholds, which was one of the starting intuitions, is not supported, with homogeneous or heterogeneous modules. A reading in terms of local traces in normal operation and a collective quantity consulted only under exception fits the ablation, but remains an interpretation.

What it is not. It is not better than the best standard non-stationary bandits, which come with theoretical guarantees (Garivier and Moulines, 2011), and it does not beat routers trained by gradient when gradients are available, whether dense or sparse; its only clear advantage there is over top-1 routing after a module dies. It does not discover structure on its own. The explanation, tested in Experiments 10 to 11, is the learning signal of the modules: the quality

of a whole path is not noisy but conjunctive, staying near zero until both modules are right at once, so modules never become primitives, and traces indexed by opaque identifiers have nothing consistent to route to. Giving each module an exact local error settles it: with local errors, traces find the shared primitives across opaque tasks, and do so with a cleaner decomposition and less interference than dense or sparse gradient routers given the same local losses. The remaining question is where a module’s local error can come from without an oracle; a module that predicts in a latent space supplies its own. Modules with their own local prediction error, for instance predictors trained in a latent space, would supply the missing signal. The difficulty of learning module layouts reported by Bahdanau et al. (2019) points in the same direction.

What it does not protect. Experiments 14b to 16b add a limit that no earlier experiment could show: nothing in the mechanism protects a function in use. Recruitment and exploration both consume a specialist once spare capacity is gone, a consumed specialist is worth no more than a fresh module, and resetting a recruited module does not help. Protection, where it was observed, came from capacity held back until an event. A body that worked like this mechanism would reassign its spinal cord to digestion at the first failure elsewhere; the mechanism allocates work, it does not preserve functions, and that distinction is the one a reviewer’s question about rare but critical skills brought out.

Against replay, and what the planning gap says about our own measures. On a released world model, under changing dynamics, the mechanism’s one structural asset, instantiating a new predictor on surprise without touching the old ones, worked as designed and was not worth more than a reservoir replay buffer with no tuned parameter. We stopped the line by the rule written before the pilot. The planning gap of Section 7 then qualifies the whole continual study: all its retention measures are in-distribution latent errors, and we now have a case where such errors, refined in every way we could think of, failed to predict control. The honest scope of Experiment 17 is therefore retention of prediction, not retention of control, and the design note’s planning phase, which we did not reach, is where the question should be settled.

Limitations. Experiments 1 to 16b come from a single toy world (five primitives in dimension 8); Experiments 12, 13 and 17 from one sequential world and one released model on one environment, with five seeds each, so their results are effect sizes with per-seed values rather than tests. The production scenarios were designed to probe resilience and are not neutral. Opponents received modest tuning budgets (8 to 20 configurations on three seeds), contextual bandits are not among them, and in Experiment 9 the bandits were not re-tuned for heterogeneous modules. Heterogeneity was limited to module size; costs, latencies and partial failures were not modelled. The decomposition of tasks is given in Experiments 1 to 6, and the local errors of Experiments 10 and 11 come from an oracle, which amounts to supervising the decomposition. Sparse routers in Experiment 11 kept the configurations of Experiment 8, not re-tuned for local losses. Modules are supervised regressors, and no real data are used. A regime in which commitment-based recruitment beats sliding-window and discounted UCB, for example with frequent failures and scarce reserves, with opponents re-tuned for that regime, remains to be identified.

9 Research integrity statement

Development seeds. Every test used seeds separate from those used to check code and tune opponents. What development runs revealed is stated in the preregistrations, including the expected failures of Experiment 7.

A protocol found after the fact. While preparing publication, the files of a preregistered protocol (Experiment 4) were found in the execution environment. They came from an earlier

execution that is absent from the history of the conversation that produced the rest of this work, and the protocol had not been reported. Its code hashes match its preregistration. Its main test had stopped at 15 of 20 seeds; seeds 95 to 99 were completed with the same code before any analysis. It is reported here in full, although it weakens an earlier claim.

Seed reuse. The preregistrations of Experiments 5 and 6 state that their seeds had never been used. Seeds 60 to 74 and 80 to 99 had in fact been used by Experiment 4. No decision depended on those results, which were unknown at the time, and the code is deterministic: the mechanism produces bit-identical results on the 20 seeds shared by two independent runs. The statement was nevertheless false, and corrections are attached to the published preregistrations.

A second set of files from an unseen execution. After the first version of this manuscript, the preregistrations and code of Experiments 8 and 9, the completed test runs of Experiment 8 and the beginning of a third, unpreregistered experiment were again found in the execution environment, from an execution absent from the conversation history. The quoted code hashes match, and the preregistrations were written before the test runs. Experiment 8 was analysed and Experiment 9 was run according to their written protocols; the third experiment was not continued and is kept as a marked draft.

A third set of files, and an overwrite. Experiment 10 (preregistration, code, complete test runs and a commit) was found in the execution environment while a second implementation of the same design was being written without knowledge of it. The second implementation briefly overwrote the first’s preregistration and code, which were restored byte for byte from the commit, and was then kept as Experiment 10b on fresh seeds.

Cause of the unseen executions, and later incidents. The files found three times came from attempts of the same conversational turns that the chat interface reported as failed while their tools kept running in the shared environment; Experiments 12 and 13 were produced the same way and completed with the same code where the attempts were cut. Later, an external review of the repository found four protocol flaws (a seed reset that did not reach the world’s own generator, recruitment rules acting before the event they were meant to answer, recruitments counted from the start of learning, reserves that were active rather than dormant) and one false claim of compute savings; each is recorded next to the result it affects, and the affected experiments were redone (15b, 14b, 16b) rather than edited. A reviewer’s non-reproduction of one run led to a perturbed replay of all 110 runs of 14b/16b: the preregistered decisions are unchanged, individual trajectories are chaotic under alarms and sampled routing.

Provenance check. A script in the repository recomputes every code hash quoted in the preregistrations and prints a ledger of seeds per result file with all overlaps between test seeds of different experiments. All hashes match, and the only overlaps are the two disclosed above.

Translation and reproducibility. The study was conducted in French. The repository keeps all files exactly as executed; the English code differs only in comments, strings and identifiers, and reproduces stored results bit for bit on nine runs covering every experiment family.

10 Conclusion

An intuition about plural, coordinated learning modules was followed, under preregistration, from a toy world to a released world model, and most of it did not survive. What held: a commitment-based backup-capacity heuristic gives resilience to module loss without a routing

gradient, at the level of good non-stationary bandits; with a local error per module, hard assignment by traces yields a cleaner decomposition than gradient mixtures. What did not: no advantage with gradients, no role for individual thresholds, no structure discovery without a local signal, no protection of functions without held-back capacity, and no advantage over replay for continual dynamics on a real representation. What was found on the way is the result we would keep if we could keep one: a predictor with ten times lower latent error that plans two times worse, and five diagnostics that exclude the obvious causes. Latent prediction error is not a measure of a world model’s worth for control, and a study that only measures it, including ours, has not measured what matters.

Code and data availability

Code, raw results, preregistrations (original French and English translations) and analysis scripts: <https://github.com/sahnas/stigmergic-module-routing>. Archived by Software Heritage with snapshot identifier `swh:1:snp:ec5e807533a26aef69001187d77b62911d3cb4c8`.

Use of AI tools

An AI system (Claude, Anthropic) acted as a research assistant throughout. The author formulated the research question and the intuitions behind the mechanism, chose which hypotheses to test or abandon, and made the decisions at each step. The AI system designed and implemented the experiments, wrote the preregistrations from the agreed hypotheses, ran the experiments and analyses on free Kaggle hardware for the world-model part, searched the literature, and drafted the manuscript; three external reviews (by other AI systems prompted by the author) found the protocol flaws recorded in Section 9. No AI system is an author.

Before submission, every reference was checked against its source, with unconfirmed bibliographic details omitted rather than guessed, and every number in this paper was recomputed from the released raw results by an independent script (`paper/check_numbers.py`). Checking numbers is not enough in such a workflow: the integrity issues of Section 9 came from the execution environment retaining files from unseen executions. Provenance, seed independence and historical completeness were therefore also checked (`tools/check_provenance.py`). The author reviewed the manuscript and takes full responsibility for its content.

References

- Dzmitry Bahdanau, Shikhar Murty, Michael Noukhovitch, Thien Huu Nguyen, Harm de Vries, and Aaron Courville. Systematic generalization: What is required and can it be learned? In *International Conference on Learning Representations*, 2019.
- Eric Bonabeau, Guy Theraulaz, and Jean-Louis Deneubourg. Quantitative study of the fixed threshold model for the regulation of division of labour in insect societies. *Proceedings of the Royal Society of London B*, 263(1376):1565–1569, 1996.
- Yang Cao, Zheng Wen, Branislav Kveton, and Yao Xie. Nearly optimal adaptive procedure with change detection for piecewise-stationary bandit. In *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pages 418–427, 2019.
- Gianni Di Caro and Marco Dorigo. AntNet: Distributed stigmergetic control for communications networks. *Journal of Artificial Intelligence Research*, 9:317–365, 1998.

- Marco Dorigo, Vittorio Maniezzo, and Alberto Colorni. Ant system: optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 26(1): 29–41, 1996.
- Jing Du, Hang Zhao, and Kenneth Huang. StigmergyRouter: A fault-aware adaptive routing demo for multi-agent AI systems. In *ACM Conference on AI and Agentic Systems (CAIS), Demonstrations*, 2026. doi: 10.1145/3786335.3813191.
- Aurélien Garivier and Eric Moulines. On upper-confidence bound policies for switching bandit problems. In *Algorithmic Learning Theory (ALT 2011)*, Lecture Notes in Computer Science, pages 174–188, 2011. doi: 10.1007/978-3-642-24412-4_16.
- Anirudh Goyal, Alex Lamb, Jordan Hoffmann, Shagun Sodhani, Sergey Levine, Yoshua Bengio, and Bernhard Schölkopf. Recurrent independent mechanisms. In *International Conference on Learning Representations*, 2021.
- Anirudh Goyal, Aniket Didolkar, Alex Lamb, Kartikeya Badola, Nan Rosemary Ke, Nasim Rahaman, Jonathan Binas, Charles Blundell, Michael Mozer, and Yoshua Bengio. Coordination among neural modules through a shared global workspace. In *International Conference on Learning Representations*, 2022.
- Pierre-Paul Grassé. La reconstruction du nid et les coordinations interindividuelles chez *Belligositermes natalensis* et *Cubitermes* sp. La théorie de la stigmergie. *Insectes Sociaux*, 6(1): 41–80, 1959.
- Masahiko Haruno, Daniel M. Wolpert, and Mitsuo Kawato. Mosaic model for sensorimotor learning and control. *Neural Computation*, 13(10):2201–2220, 2001.
- Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.
- Michael J. B. Krieger, Jean-Bernard Billeter, and Laurent Keller. Ant-like task allocation and recruitment in cooperative robots. *Nature*, 406:992–995, 2000.
- Anson Lei, Frederik Nolte, Bernhard Schölkopf, and Ingmar Posner. Compete and compose: Learning independent mechanisms for modular world models. *arXiv preprint arXiv:2404.15109*, 2024.
- Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworld-model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.
- Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.
- Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018.
- Jonas Pfeiffer, Sebastian Ruder, Ivan Vulić, and Edoardo Maria Ponti. Modular deep learning. *Transactions on Machine Learning Research*, 2023.
- Clemens Rosenbaum, Tim Klinger, and Matthew Riemer. Routing networks: Adaptive selection of non-linear functions for multi-task learning. In *International Conference on Learning Representations*, 2018.

- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.
- Jekanthan Thangavelautham and Gabriele M. T. D’Eleuterio. Tackling learning intractability through topological organization and regulation of cortical networks. *IEEE Transactions on Neural Networks and Learning Systems*, 2012. doi: 10.1109/TNNLS.2011.2178311.
- Guy Theraulaz, Eric Bonabeau, and Jean-Louis Deneubourg. Response threshold reinforcements and division of labour in insect societies. *Proceedings of the Royal Society of London B*, 265 (1393):327–332, 1998. doi: 10.1098/rspb.1998.0299.
- Xudong Wang, Chaoning Zhang, Jiaquan Zhang, Chenghao Li, Qigan Sun, Sung-Ho Bae, Peng Wang, Ning Xie, Jie Zou, Yang Yang, and Hengtao Shen. Efficient and interpretable multi-agent LLM routing via ant colony optimization. *arXiv preprint arXiv:2603.12933*, 2026.
- Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.
- Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.